

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	M Rakesh	Reg. No.:	192421006
Section / Batch:	IT	Date:	8/1/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1, Debugging Insertion Sort Code (Wrong While Condition Uses Inverted Order in C)

The following Insertion Sort implementation in C uses the wrong comparison direction in the while-loop, causing it to shift smaller elements instead of larger ones and produce descending-like behavior in parts of the array. Carefully analyze how the sorted prefix invariant is broken by the inverted condition. Apply systematic debugging so that only elements greater than the key are

shifted to the right before insertion. Optimize the corrected implementation for readability and consistent indexing. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean C code with proper comments.

```
void insertionSort(int arr[], int n) {
    int i, j, key;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] < key) { // ERROR
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Corrected Insertion Sort

```
#include <stdio.h>
```

```
void insertionSort(int arr[], int n)
{
    int i, j, key;

    for(i = 1; i < n; i++)
    {
        key = arr[i];
        j = i - 1;
```

```

        while(j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}

```

```

void printArray(int arr[], int n)
{
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);
}

```

```

int main()
{
    int arr[] = {12, 11, 13, 5, 6};
    int n = sizeof(arr) / sizeof(arr[0]);

    insertionSort(arr, n);

    printArray(arr, n);

    return 0;
}

```

Error: arr[j] < key should be arr[j] > key.

Time Complexity:

- Best Case: **$O(n)$**
- Average Case: **$O(n^2)$**
- Worst Case: **$O(n^2)$**
- Space Complexity: **$O(1)$**

2, Debugging Linear Search Code (Search Continues but Result Is Overwritten Incorrectly)

The following Linear Search implementation tries to record the location of the key, but it overwrites the result even after a successful match because of an incorrect else branch. Carefully analyze the control flow and explain why a later non-matching element can erase a previously found answer. Apply systematic debugging so that the result remains valid once the key is found. Optimize the implementation for simplicity and immediate termination on success. Analyze the time complexity of the corrected solution and rewrite the final code with suitable comments.

```
def linear_search(arr, key):
    pos = -1
    for i in range(len(arr)):
        if arr[i] == key:
            pos = i
        else:
            pos = -1 # ERROR
    return pos
```

Corrected Linear Search

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1
```

```
arr = [10, 20, 30, 40, 50]
```

```
key = 30
```

```
result = linear_search(arr, key)
```

```
if result != -1:
```

```
    print("Element found at index", result)
```

```
else:
```

```
    print("Element not found")
```

Error: else: pos = -1 overwrites the correct position after a match.

Time Complexity:

- Best Case: **O(1)**
- Average Case: **O(n)**
- Worst Case: **O(n)**
- Space Complexity: **O(1)**